

LAPORAN PRAKTIKUM MINGGU 5

Topik: Abstraction (Abstract Class & Interface) Implementasi Pembayaran

A. IDENTITAS

Nama	:	Sri Wahyuningsih
Nim	:	240202844
Kelas	:	3IKRA

B. TUJUAN

Adapun tujuan dari praktikum ini adalah agar mahasiswa mampu:

1. Memahami konsep abstract class dan interface di Java.
2. Mengimplementasikan metode pembayaran Cash, EWallet, dan TransferBank menggunakan abstraction dan interface.
3. Mempelajari multiple inheritance via interface untuk validasi dan pencetakan struk.

C. DASAR TEORI

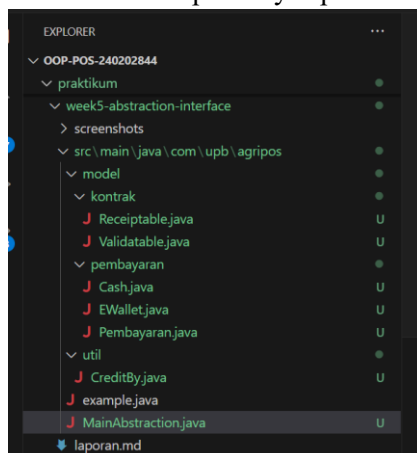
Adapun dasar teori yang mendasari praktikum ini, di antaranya:

1. Abstract class adalah kelas yang tidak bisa diinstansiasi dan dapat memiliki method abstrak maupun konkrit.
2. Interface adalah kontrak method yang harus diimplementasikan kelas, mendukung multiple inheritance.
3. Polimorfisme memungkinkan objek dari subclass diperlakukan sebagai objek dari superclass.
4. Validasi dan pencetakan struk dapat diimplementasikan menggunakan interface agar fleksibel.
5. Multiple inheritance via interface aman karena interface hanya mendefinisikan kontrak, bukan implementasi.

D. LANGKAH PRAKTIKUM

1. Setup Project:

Buat struktur repository seperti berikut:



2. Coding

- a. Buat Abstract Class Pembayaran
 - Field: invoiceNo, total

- Method abstrak:
 - double biaya() → menghitung biaya tambahan/fee
 - boolean prosesPembayaran() → status berhasil/gagal
 - Method konkret:
 - double totalBayar() → total + biaya()
- b. Buat Subclass Konkret
- Cash:
 - biaya() = 0
 - prosesPembayaran() → selalu berhasil jika tunai $\geq$ totalBayar()
 - Implementasikan interface Receiptable
 - EWallet:
 - biaya() = 2000
 - prosesPembayaran() → memerlukan validasi OTP (Validatable)
 - Implementasikan Validatable & Receiptable
 - TransferBank (Latihan Mandiri):
 - biaya() = 2500
 - prosesPembayaran() → memerlukan validasi (Validatable)
 - Implementasikan Validatable & Receiptable
- c. Buat Interface
- Validatable: boolean validasi()
 - Receiptable: String cetakStruk()
- d. Main Class
- Buat MainAbstraction.java
 - Tampilkan struk pembayaran polimorfik untuk Cash, EWallet, dan TransferBank
 - Panggil CreditBy.print(); di akhir untuk identitas mahasiswa
4. Commit & Push:
- Commit dengan pesan: week5-abstraction-interface

E. KODE PROGRAM

1. Pembayaran.java

```

Pembayaran.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripis > model > pembayaran > J Pembayaran
1 package com.upb.agripis.model.pembayaran;
2
3 public abstract class Pembayaran {
4     protected String invoiceNo;
5     protected double total;
6
7     public Pembayaran(String invoiceNo, double total) {
8         this.invoiceNo = invoiceNo;
9         this.total = total;
10    }
11
12    public abstract double biaya(); // biaya tambahan/admin
13    public abstract boolean prosesPembayaran(); // proses spesifik tiap metode
14
15    public double totalBayar() {
16        return total + biaya();
17    }
18
19    public String getInvoiceNo() { return invoiceNo; }
20    public double getTotal() { return total; }
21 }
22

```

2. Validatable.java

```

Validatable.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripis
1 package com.upb.agripis.model.kontrak;
2
3 public interface Validatable {
4     boolean validasi(); // misal validasi OTP/PIN
5 }
6

```

3. Receiptable.java

```
J Receiptable.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripos > model > kontrak > Receiptable.java
1 package com.upb.agripos.model.kontrak;
2
3 public interface Receiptable {
4     String cetakStruk();
5 }
6
```

4. Cash.java

```
J Cash.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripos > model > pembayaran > Cash.java
1 package com.upb.agripos.model.pembayaran;
2 import com.upb.agripos.model.kontrak.Receiptable;
3
4 public class Cash extends Pembayaran implements Receiptable {
5     private double tunai;
6
7     public Cash(String invoiceNo, double total, double tunai) {
8         super(invoiceNo, total);
9         this.tunai = tunai;
10    }
11
12    @Override
13    public double biaya() {
14        return 0.0;
15    }
16
17    @Override
18    public boolean prosesPembayaran() {
19        return tunai >= totalBayar();
20    }
21
22    @Override
23    public String cetakStruk() {
24        return String.format(
25            "FORMAT INVOICE No | TOTAL: %.0f | BAYAR CASH: %.0f | KEMBALI: %.0f",
26            invoiceNo, total, tunai, Math.max(0, tunai - totalBayar());
27        );
28    }
29 }
```

5. EWallet.java

```
J EWallet.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripos > model > pembayaran > EWallet.java
1 package com.upb.agripos.model.pembayaran;
2 import com.upb.agripos.model.kontrak.Validatable;
3 import com.upb.agripos.model.kontrak.Receiptable;
4
5 public class EWallet extends Pembayaran implements Validatable, Receiptable {
6     private String akun;
7     private String otp;
8     private static final double FEE = 2000;
9
10    public EWallet(String invoiceNo, double total, String akun, String otp) {
11        super(invoiceNo, total);
12        this.akun = akun;
13        this.otp = otp;
14    }
15
16    @Override
17    public double biaya() {
18        return FEE;
19    }
20
21    @Override
22    public boolean validasi() {
23        return otp != null && otp.length() == 6;
24    }
25 }
```

```
27 @Override
28 public boolean prosesPembayaran() {
29     return validasi();
30 }
31
32 @Override
33 public String cetakStruk() {
34     return String.format(
35         "FORMAT INVOICE No | TOTAL: %.0f | BAYAR E-WALLET: %.0f | STATUS: %s",
36         invoiceNo, totalBayar(), total, biaya(), akun, prosesPembayaran() ? "BERHASIL" : "GAGAL"
37     );
38 }
39 }
```

6. CreditBy.java

```
J CreditBy.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripos > util > CreditBy.java
1 package com.upb.agripos.util;
2
3 public class CreditBy {
4     private static final String NAMA = "Sriwas";
5     private static final String NIM = "240202044";
6
7     public static void print() {
8         System.out.println("Credit by " + NAMA + " | NIM: " + NIM);
9     }
10 }
11
```

7. MainAbstraction.java

```
J MainAbstraction.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripos > MainAbstraction.java
1 package com.upb.agripos;
2 import com.upb.agripos.model.pembayaran.*;
3 import com.upb.agripos.model.kontrak.*;
4 import com.upb.agripos.util.CreditBy;
5
6 public class MainAbstraction {
7     public static void main(String[] args) {
8         Pembayaran cash = new Cash(invoiceNo:"MKA-001", total:20000, tunai:20000);
9         System.out.println(((Receiptable) cash).cetakStruk());
10
11         Pembayaran ewallet = new EWallet(invoiceNo:"MKA-001", total:221300, akun:"sawajewallet", otp:"820005");
12         System.out.println(((Receiptable) ewallet).cetakStruk());
13
14         Pembayaran transfer = new TransferBank(invoiceNo:"MKA-001", total:20000, akunBank:"wasapbank", otp:"112300");
15         System.out.println(((Receiptable) transfer).cetakStruk());
16
17         CreditBy.print();
18     }
19 }
20
21
```

8. TransferBank.java

```
J TransferBank.java U X
praktikum > week5-abstraction-interface > src > main > java > com > upb > agripos > model > pembayaran > TransferBank.java
1 package com.upb.agripos.model.pembayaran;
2 import com.upb.agripos.model.kontrak.Validatable;
3 import com.upb.agripos.model.kontrak.Receiptable;
4
5 public class TransferBank extends Pembayaran implements Validatable, Receiptable {
6     private String akunBank;
7     private static final double BAWA TRANSFER = 1000;
8
9     public TransferBank(String invoiceNo, double total, String akunBank, String otp) {
10         super(invoiceNo, total);
11         this.akunBank = akunBank;
12         this.otp = otp;
13     }
14
15     @Override
16     public double biaya() {
17         return BAWA_TRANSFER;
18     }
19
20     @Override
21     public boolean validasi() {
22         return otp != null && otp.length() == 6;
23     }
24 }
```

```
21 @Override
22 public boolean validasi() {
23     return otp != null && otp.length() == 6;
24 }
25
26 @Override
27 public boolean prosesPembayaran() {
28     return validasi();
29 }
30
31 @Override
32 public String cetakStruk() {
33     return String.format(
34         "FORMAT INVOICE No | TOTAL: %.0f | BAYAR TRANSFER BANK: %.0f | STATUS: %s",
35         invoiceNo, totalBayar(), total, biaya(), akunBank, prosesPembayaran() ? "BERHASIL" : "GAGAL"
36     );
37 }
38 }
```

F. HASIL EKSEKUSI

```
PS C:\Users\sri61\Documents\oop-pos-240202844\praktikum\week5-abstraction-interface\src\main\java> java MainAbstraction
● raction
INVOICE WWA-001 | TOTAL: 200000 | BAYAR CASH: 250000 | KEMBALI: 50000
INVOICE WWA-002 | TOTAL+FEE: 223300 (221300 + 2000) | E-WALLET: wawa@ewallet | STATUS: BERHASIL
INVOICE WWA-003 | TOTAL+BIAYA: 202500 (200000 + 2500) | TRANSFER BANK: wawa@bank | STATUS: BERHASIL
Credit by Sriwaa | NIM: 240202844
PS C:\Users\sri61\Documents\oop-pos-240202844\praktikum\week5-abstraction-interface\src\main\java>
```

G. ANALISIS

- a. Cara kode berjalan:
 - Program menggunakan abstract class Pembayaran sebagai superclass yang menyediakan struktur dasar (invoiceNo, total, dan method totalBayar()).
 - Subclass (Cash, EWallet, TransferBank) mengimplementasikan method abstrak biaya() dan prosesPembayaran() sesuai karakteristik masing-masing.
 - Interface (Validatable & Receiptable) digunakan untuk menambahkan fitur validasi dan cetak struk tanpa mempengaruhi hierarchy class.
 - Di MainAbstraction.java, objek dibuat sebagai Pembayaran (polimorfik) tetapi tetap bisa dipanggil method cetakStruk() melalui casting ke Receiptable, sehingga polimorfisme dan dynamic binding berjalan.
- b. Perbedaan pendekatan dengan minggu sebelumnya:
 - Minggu sebelumnya fokus pada class & object sederhana tanpa abstraction.
 - Minggu ini menggunakan abstract class untuk perilaku umum dan interface untuk kontrak tambahan, sehingga program lebih modular, fleksibel, dan mudah dikembangkan.
 - Memungkinkan menambahkan metode pembayaran baru (misal TransferBank) tanpa merusak kode lama.
- c. Kendala dan solusi:
 - Awalnya bingung cara implementasi multiple inheritance → diselesaikan dengan interface (Validatable & Receiptable).
 - Mengatur perhitungan total+fee/biaya supaya struk tetap ringkas → diselesaikan dengan menambahkan method biaya() di masing-masing subclass dan menampilkan (total + fee) di struk.
 - Casting ke Receiptable di MainAbstraction diperlukan agar bisa memanggil cetakStruk() pada objek Pembayaran.
- d. Kelebihan pendekatan ini:
 - Lebih mudah menambahkan metode pembayaran baru.
 - Polimorfisme dan dynamic binding memudahkan menampilkan struk untuk berbagai metode pembayaran.
 - Struktur kode lebih jelas: abstract class untuk perilaku dasar, interface untuk fitur tambahan.

H. KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Dengan menggunakan abstract class, program dapat mendefinisikan struktur dan perilaku dasar yang sama untuk semua metode pembayaran, sekaligus memaksa subclass untuk mengimplementasikan method khusus mereka sendiri.
2. Interface memungkinkan penambahan fitur tambahan seperti validasi OTP (Validatable) dan cetak struk (Receiptable) tanpa mengganggu hierarki class, sehingga multiple inheritance bisa dilakukan dengan aman.

3. Program menjadi lebih modular, fleksibel, dan mudah dikembangkan, karena metode pembayaran baru dapat ditambahkan tanpa merusak kode yang sudah ada.
4. Polimorfisme dan dynamic binding memungkinkan penggunaan objek subclass sebagai superclass (Pembayaran) sambil tetap dapat memanggil fitur spesifik melalui interface, sehingga menampilkan struk pembayaran secara konsisten untuk berbagai metode.

QUIZ

1. Jelaskan perbedaan konsep dan penggunaan abstract class dan interface.

Jawaban:

- Abstract class bisa memiliki field, constructor, method abstrak, dan method konkret. Digunakan ketika beberapa subclass memiliki sifat dan perilaku dasar yang sama, tapi implementasinya berbeda.
- Interface hanya mendefinisikan kontrak method tanpa state, digunakan untuk menjamin bahwa kelas yang mengimplementasi memiliki method tertentu. Interface mendukung multiple inheritance secara aman.

2. Mengapa multiple inheritance lebih aman dilakukan dengan interface pada Java?

Jawaban:

Java tidak mendukung multiple inheritance dengan class karena bisa terjadi diamond problem (method sama dari superclass berbeda).

Dengan interface, kelas bisa mengimplementasikan beberapa kontrak sekaligus tanpa konflik implementasi, karena interface hanya mendefinisikan method abstrak, bukan implementasi.

Contoh: EWallet mengimplementasikan Validatable dan Receiptable sekaligus tanpa masalah.

3. Pada contoh Agri-POS, bagian mana yang paling tepat menjadi abstract class dan mana yang menjadi interface? Jelaskan alasannya.

Jawaban:

- Abstract class: Pembayaran, karena semua metode pembayaran memiliki field invoiceNo dan total serta method dasar totalBayar(), namun implementasi biaya() dan prosesPembayaran() berbeda pada setiap subclass.
- Interface: Validatable dan Receiptable, karena beberapa metode pembayaran memerlukan fitur tambahan seperti validasi OTP atau cetak struk. Dengan interface, multiple inheritance dapat dilakukan secara aman.

CHECKLIST KEBERHASILAN

- | | |
|---|---|
| ✓ | Abstract class Pembayaran memiliki method abstrak dan method konkret yang tepat. |
| ✓ | Interface diimplementasikan dengan benar pada kelas yang relevan. |
| ✓ | Multiple inheritance via interface berjalan (kelas mengimplementasikan ≥ 2 interface). |
| ✓ | Program menampilkan struk dan status proses pembayaran. |
| ✓ | Output menyertakan credit by: - melalui CreditBy. |
| ✓ | Screenshot & laporan telah dilampirkan. |